ELAW 프로젝트

프로젝트 진행 과정 종합 보고서

DB/AI 담당자 관점의 전체 작업 정리 및 프로젝트 영향 분석

DB/AI 담당 오 | 2026년 3월 | ELAW v1.0

항목	내용
문서 목적	ELAW 프로젝트 DB/AI 영역의 전체 진행 과정과 산출물, 프로젝트 내 역할을 종합 정리
작성 기간	2026년 3월 (약 3개월 개발 기간 중 Sprint 1~6 완료 시점)
대상 독자	팀원 전체 (팀장 김태, 백엔드 김건, 프론트엔드 윤, DB/AI 오) 및 지도 교수
핵심 성과	21개 파일 · 5,700줄+ 코드 · 39개 테스트 100% 통과 · 버그 22개 발견·수정

1. 프로젝트 개요 및 목표

ELAW(AI 기반 학습·문제풀이 이력 통합 포트폴리오 및 취업 매칭 플랫폼)는 개발자 지망생의 분산된 학습 이력을 한 곳에 모으고, AI로 분석해 신뢰할 수 있는 포트폴리오를 자동 생성하며, 기업 채용 공고와 데이터 기반으로 매칭해주는 통합 플랫폼이다.

1.1 해결하는 문제

- 학습 데이터 단편화 — 백준, 프로그래머스, W3Schools 등 여러 플랫폼에 흩어진 이력을 통합할 창구 없음
- 기업 평가 한계 — 이력서·면접 중심의 주관적 평가, 실무 역량 정량 지표 부재
- 개인화 추천 부재 — 다중 플랫폼 데이터 기반 맞춤 커리큘럼 제공 불가
- 교육-채용 연계 단절 — 학습 결과가 채용 과정으로 자연스럽게 이어지지 않음

1.2 팀 구성 및 역할

역할	담당자	주요 책임
팀장	김태	전체 일정·아키텍처 총괄, 요구사항 명세, 코드 리뷰
백엔드	김건	Django API 서버, JWT 인증, ETL 배치 스케줄러, orion 서버 인프라
프론트엔드	윤	HTML/CSS/JS UI, 대시보드·포트폴리오 화면, 반응형 레이아웃
DB/AI	오	MySQL 스키마, ORM 모델, Gemini AI 모듈, ETL 파이프라인, 캐시·매칭 엔진

1.3 기술 스택

레이어	기술	선택 이유
백엔드	Python 3.12 + Django 4.2	ORM·Admin·인증 내장, 팀 학습 부담 최소화
데이터베이스	MySQL 8.x	관계형 모델로 정규화, JSON 컬럼 지원
AI	Gemini 1.5 Pro API	커리큘럼 생성·포트폴리오·매칭 분석
캐시	Redis 7	Gemini 응답 캐싱(TTL), 토큰 블랙리스트
인프라	Docker Compose / orion 서버	로컬 개발 Docker, 배포 Ubuntu + Nginx
인증	JWT (Access 30분 / Refresh 7일)	Stateless API, Role 기반 접근 제어

2. 전체 진행 과정 (5단계)

DB/AI 담당자가 수행한 작업을 Sprint 순서에 따라 5단계로 정리한다. 각 단계의 목적, 수행 내용, 산출물, 그리고 발견·수정한 버그를 함께 기술한다.

1 기획 및 설계

2026년 3월 초 · Sprint 0

- 제품 기획서 2개(기획서1·기획서2) 분석 및 통합 — 중복 내용 제거, 누락 기능 보완
- PRD v1.0 작성 — 12개 섹션, 기능 요구사항 40개, P0/P1/P2 우선순위 분류
- DB 담당자 역할 정의서 작성 — 20개 작업 항목, Sprint별 체크리스트, 협업 인터페이스
- 산출물: ELAW_PRD.docx, ELAW_DB_Role.docx

2 DB 스키마 설계 (Sprint 1~2)

2026년 3월 · P0 작업

- MySQL 스키마 설계 — 12개 테이블, 2개 뷰, 인덱스 전략 수립 (schema_v1.sql · 527줄)
- Django ORM 모델 작성 — 12개 모델, managed=False 설정, 관계 정의 (models.py · 451줄)
- ERD DBML 작성 — dbdiagram.io 붙여넣기용 전체 관계 명세 (erd_v1.dbml · 240줄)
- 시드 데이터 구축 — users 15명, curricula 10건(JSON), job_postings 6건, matches 20건, solve_history 204건
- 더미 데이터 생성 스크립트 작성 (generate_seed.py · 197줄) — 학생별 취약 태그 패턴 반영
- 버그 수정 ①: ENUM 안 인라인 주석(--) 8곳 → MySQL 파싱 오류, 전량 제거
- 버그 수정 ②: managed=False 누락 → migrate 시 table already exists 충돌, 12개 모델 추가
- 버그 수정 ③: 비밀번호 PLACEHOLDER 해시 15개 → SHA-256 DEV 해시 60자 교체
- 산출물: schema_v1.sql, models.py, erd_v1.dbml, seed_data.sql, generate_seed.py

3 AI 모듈 + ETL 파이프라인 (Sprint 3~4)

2026년 4월 · P1 작업

- Gemini AI 모듈 개발 (gemini_client.py · 768줄) — 5개 기능 구현
- · 커리큘럼 생성: 목표+학습이력 → 주차별 JSON / 취약점 분석: algo_tag 정답률 → Top-5 약점+조언
- · 포트폴리오 생성: 풀이이력+GitHub → 섹션별 서술 / 채용 매칭: 역량↔공고 0~100점
- · 문제 추천: 취약 태그 기반 백준 문제 3~5개 / 재시도 3회, JSON 파싱, ai_logs 저장
- ETL 파이프라인 개발 (platform_collector.py · 669줄)
- · BaekjoonExtractor: solved.ac API 호출, GitHubExtractor: REST API v3

- · DataTransformer: 언어명 정규화(Python3→Python), 태그 한국어화, tier→난이도 변환
- · DataLoader: INSERT IGNORE, UPSERT, learning_stats 집계 배치
- Redis 캐시 레이어 (redis_cache.py · 351줄) — 7개 키 패턴, TTL 관리, 토큰 블랙리스트
- 매칭 엔진 (match_engine.py · 372줄) — 규칙기반 60점+Gemini 40점 2단계 점수 계산
- 버그 수정 ④: _load_profile user_id 무관 동일 Mock → user_id 1·2·3 별도 분기
- 버그 수정 ⑤: language 기본값 'Unknown' → 빈 문자열·None 처리
- 버그 수정 ⑥: flush_user가 curriculum:{uid}:{hash} 키 삭제 불가 → MockRedis keys() 스캔
- 산출물: gemini_client.py, platform_collector.py, redis_cache.py, match_engine.py

4 통합 테스트 + 검증 (Sprint 5~6)

2026년 5월 · P1~P2 작업

- 39개 테스트 스위트 작성 (test_db_ai.py · 698줄) — 스키마·시드·AI·ETL·매칭·캐시 전 영역
- 배치 스케줄 — cron 7개 자동 등록 (cron_setup.sh · 62줄)
- · 새벽 3시: 백준/GitHub 수집 / 새벽 4시: learning_stats 재집계 / 자정: 만료공고 처리
- · 주 1회: 매칭점수 재계산 / 새벽 2시: DB 백업 / 새벽 9시: AI 비용 리포트
- MySQL 백업 스크립트 (backup_mysql.sh · 58줄) — mysqldump gzip, 7일 자동 삭제
- Django management commands 5개 — sync_platforms, rebuild_stats, expire_postings, run_matching_batch, ai_cost_report
- 인덱스 분석 SQL (index_analysis.sql · 194줄) — 6개 주요 쿼리 EXPLAIN 검증
- CRITICAL 버그 수정 ⑦: schema_v1.sql ENUM 주석 MySQL 파싱 오류 → 완전 제거
- CRITICAL 버그 수정 ⑧: backup_mysql.sh 줄이음(\) 뒤 공백+주석 → bash 오류, 재작성
- HIGH 버그 수정 ⑨: management_commands import 경로 (from etl. → from core.etl.)
- SETUP_GUIDE.md, requirements.txt, settings_example.py, env_example.txt 작성
- 최종 결과: 39/39 통과, bash 문법 오류 0개
- 산출물: test_db_ai.py, backup_mysql.sh, cron_setup.sh, management_commands.py, 가이드 문서 4개

5 로컬 환경 구성 + 실행 (Windows 11)

2026년 3월 · 실제 구동

- Docker Desktop 설치 → docker-compose.yml 작성 → MySQL+Redis+Python 컨테이너 구성
- 오류 해결 12회 — SSL 인증서 오류(--ssl=0), nano 없음(sed 대체), 경로 불일치, 모듈 누락 등
- 폴더 구조 정리 — 중복 폴더(ai_module/, etl/, migrations/) 제거, import 경로 수정
- 시드 데이터 204건 입력 (pymysql 직접 삽입으로 SQL 청크 오류 우회)
- 환경변수(.env) 설정 — SECRET_KEY 생성, GEMINI_API_KEY 발급 및 적용
- 최종 테스트: PYTHONPATH=/app python3 tests/test_db_ai.py → 39/39 통과

- 현재 상태: 로컬 개발 환경 완성, Django 서버 실행 가능 (http://localhost:8000)

3. 전체 산출물 목록

Sprint 1~6를 통해 생성된 전체 파일 목록과 각 파일의 역할을 정리한다.

파일명	Sprint	역할	코드량
schema_v1.sql	1	MySQL 12개 테이블 + 2개 뷰 DDL	527줄
models.py	1	Django ORM 모델 12개 (managed=False)	451줄
erd_v1.dbml	1	ERD DBML — dbdiagram.io 붙여넣기용	240줄
seed_data.sql	2	개발용 시드 데이터 (users 15명 등)	413줄
generate_seed.py	2	풀이 이력 대량 생성 스크립트	197줄
seed_solve_history.sql	2	generate_seed.py 출력 SQL	393줄
gemini_client.py	3	Gemini API 5개 기능 모듈	768줄
platform_collector.py	3	백준·GitHub ETL 파이프라인	669줄
redis_cache.py	3	Redis 캐시 레이어 (7개 키 패턴)	351줄
match_engine.py	5	2단계 채용 매칭 점수 엔진	372줄
test_db_ai.py	6	39개 통합 테스트 스위트	698줄
backup_mysql.sh	6	MySQL 백업 스크립트 (gzip, 7일)	58줄
cron_setup.sh	6	cron 7개 배치 자동 등록	62줄
management_commands.py	6	Django commands 5개 코드	299줄
index_analysis.sql	5	인덱스 EXPLAIN 분석 쿼리	194줄
SETUP_GUIDE.md	6	orion 서버 배포 가이드	629줄
requirements.txt	6	Python 패키지 의존성 (버전 상한 포함)	33줄
settings_example.py	6	Django settings.py 예시	121줄
env_example.txt	6	.env 환경변수 예시	31줄
README.md	6	전체 모듈 사용 가이드	125줄
erd_v1.dbml	1	ERD DBML 관계 명세	240줄

핵심 수치

총 파일: 21개 | 총 코드량: 5,700줄+ | 39개 테스트 100% 통과

4. 발견·수정한 버그 전체 목록

두 차례의 검토(Sprint 완료 후 + 실행 가이드 검증)를 통해 총 22개 버그를 발견하고 전량 수정하였다.

등급	파일	버그 내용	수정 방법
CRITICAL	schema_v1.sql	ENUM 안 -- 인라인 주석 8곳 → MySQL 파싱 오류로 테이블 생성 실패	ENUM 블록 내 주석 전량 제거 (파이썬 파서로 자동 처리)
CRITICAL	backup_mysql.sh	mysqldump 줄이음(\) 뒤 공백+주석 → bash 파싱 오류	주석을 블록 주석으로 분리, 스크립트 전면 재작성
HIGH	models.py	managed=False 누락 → migrate 시 'table already exists' 오류	12개 모델 Meta 클래스에 managed=False 일괄 추가
HIGH	platform_collector.py	management command 템플릿 'from etl.' → Django 배포 시 ModuleNotFoundError	'from core.etl.' 로 전체 교체
HIGH	management_commands.py	'from etl.', 'from matching.' → Django import 실패	'from core.etl.', 'from core.matching.' 로 수정
MEDIUM	SETUP_GUIDE.md	테스트 경로 'test/' 만 표기, Django 이동 후 'tests/' 경로 불일치	두 경로 모두 명시, 상황별 안내 분리
MEDIUM	SETUP_GUIDE.md	작업 기준 디렉토리 불명확 (elaw_db/ vs /srv/elaw/ 혼재)	상단에 '작업 디렉토리 기준 안내' 박스 추가
MEDIUM	requirements.txt	celery(선택)가 필수 목록 포함, apt 사전 설치 미안내	requirements-optional.txt 분리, apt 안내 추가
MEDIUM	seed_data.sql	비밀번호 PLACEHOLDER 해시 15개 실제 동작 불가	SHA-256 DEV 해시 60자 고정으로 전체 교체
LOW	requirements.txt	버전 상한 없음(>=만) → 메이저 업그레이드 시 호환성 깨짐	모든 패키지 ',<X.0.0' 상한 추가
LOW	gemini_client.py	_load_profile이 user_id 무관 동일 Mock → 테스트 신뢰도 저하	user_id 1·2·3 별도 프로필, 미등록 ID 기본값 분기
LOW	platform_collector.py	language 기본값 'Unknown' 고정 → 의미없는 값 DB 저장	빈 문자열이면 None 처리 (clean_language 변수)
LOW	platform_collector.py	solved_at이 항상 수집 시각으로 덮어씀	solved_at 파라미터 추가, 외부 제공 시 해당 값 우선
LOW	redis_cache.py	flush_user가 curriculum:{uid}:{hash} 키 삭제 불가	MockRedis keys() 전체 스캔 후 패턴 매칭 삭제
LOW	redis_cache.py	health_check 예외 처리 범위 불완전	AttributeError 별도 처리, 실패 시 _available=False
LOW	match_engine.py	_save_match Mock 여부 로그에서 불명확	로그에 '[Mock]' 명시, Django ORM 구현 주석 보강

등급	파일	버그 내용	수정 방법
LOW	schema_v1.sql	DROP 목록에 schema_versions, 뷰 누락	DROP VIEW 2개 + DROP TABLE schema_versions 추가
LOW	schema_v1.sql	time_spent_min SMALLINT → 음수 저장 가능	SMALLINT UNSIGNED으로 변경
LOW	schema_v1.sql	v_user_solve_summary 뷰 LEFT JOIN 시 NULL SUM	COALESCE + CASE WHEN COUNT=0 THEN NULL 처리
LOW	test_db_ai.py	기존 26개 테스트 → 버그 발견 후 13개 추가 필요	새 테스트 13개 추가 → 총 39개
LOW	etl	generate_seed.py SQL 청크 세미콜론 오류로 일부 데이터 미삽입	pymysql 직접 삽입 방식으로 우회 (로컬 실행 시)
LOW	import 경로	tests/test_db_ai.py가 'matching.' 등 구 경로 참조	sed로 'core.ai.', 'core.etl.' 등으로 일괄 수정

5. 전체 프로젝트에서의 영향

DB 담당자(오)가 완성한 작업이 ELAW 프로젝트 전체 시스템에 어떻게 연결되는지, 각 팀원에게 어떤 영향을 주는지 정리한다.

5.1 시스템 전체 구조에서의 위치

ELAW 플랫폼은 5개 레이어로 구성된다. DB 담당자가 완성한 부분은 아래 표의 음영 영역이다.

레이어	담당	완성 여부	내용
사용자 레이어	전체	—	학생 / 기업 / 관리자 3가지 역할
프론트엔드	윤	진행 중	HTML+CSS+JS 대시보드, 포트폴리오, 취업 UI
백엔드 API	김건	진행 중	Django RESTful API, JWT 인증, URL 라우팅
DB/AI/ETL/캐시	오 (본 문서)	완료	스키마·ORM·AI·ETL·캐시·매칭 전체
인프라	김건+오	구성 완료	Docker Compose(로컬) / orion 서버(배포 예정)

5.2 팀원별 영향

백엔드 담당자 (김건)에게 전달되는 것

schema_v1.sql + models.py — Django에서 바로 ORM으로 쿼리 가능. migrate --fake-initial 한 번이면 연결 완료

gemini_client.py — AiService().get_curriculum(user_id, goal_id) 한 줄 호출로 AI 기능 사용 가능

platform_collector.py — python manage.py sync_platforms 로 백준·GitHub 데이터 자동 수집

match_engine.py — MatchEngine().run_all_postings(user_id) 로 채용 매칭 점수 일괄 계산

management commands 5개 — Django에 등록하면 cron 배치 즉시 사용 가능

프론트엔드 담당자 (윤)에게 전달되는 것

대시보드 API 응답 JSON 구조 — { labels: ['Python','Java',...], values: [90,80,...] } 형태로 설계됨

포트폴리오 공개 슬러그 — /portfolio/{public_slug} 경로로 공개 URL 바로 사용 가능

learning_stats 테이블 — 레이더 차트, 꺾은선 그래프 등 시각화에 필요한 집계 데이터 제공

matches 테이블 — match_score DESC로 정렬된 추천 공고 리스트 바로 활용 가능

팀장 (김태)에게 전달되는 것

39개 자동 테스트 — python3 tests/test_db_ai.py 한 줄로 전체 DB/AI 모듈 정상 여부 즉시 확인

배치 스케줄 7개 — ./batch/cron_setup.sh 실행 후 새벽 자동 수집·집계·백업 가동
AI 비용 리포트 — python manage.py ai_cost_report --days 7 로 Gemini API 비용 추적
index_analysis.sql — EXPLAIN 분석으로 쿼리 성능 병목 발생 시 즉시 진단 가능

5.3 다음 단계 — 백엔드 팀이 해야 할 일

DB 모듈이 완성되었으므로, 백엔드 담당자(김건)가 아래 순서로 API 개발을 시작할 수 있다.

1. Django 프로젝트에 core/models.py 적용 → python manage.py migrate --fake-initial
2. INSTALLED_APPS에 'core' 추가, config/urls.py에 API 라우팅 연결
3. views.py에서 AiService, ETLPipeline, MatchEngine 임포트 후 API 엔드포인트 구현
4. JWT 인증 미들웨어 적용 (djangorestframework-simplejwt 설치)
5. management commands 5개를 core/management/commands/에 배포 후 cron 등록

5.4 현재 실행 가능한 것

기능	명령어	상태
39개 단위 테스트	PYTHONPATH=/app python3 tests/test_db_ai.py	완료 (39/39)
Django 서버 실행	python manage.py runserver 0.0.0.0:8000	완료
DB 연결 확인	python manage.py dbshell	완료
백준 데이터 수집 (Mock)	python manage.py sync_platforms --user 1	완료
학습 통계 재집계	python manage.py rebuild_stats	완료
AI 비용 리포트	python manage.py ai_cost_report	완료
cron 배치 등록	./batch/cron_setup.sh	완료

6. 회고 및 교훈

6.1 잘된 점

- 39개 테스트를 먼저 작성하고 코드를 검증하는 방식 덕분에 CRITICAL 버그 2개를 배포 전에 발견함
- Docker Compose로 로컬 환경을 구성하여 orion 서버 없이도 전체 개발·테스트가 가능해짐
- managed=False 패턴으로 SQL 스키마와 Django ORM을 동시에 유지하는 구조가 협업에 효과적
- 파일마다 배포 경로 주석과 활성화 코드를 함께 제공하여 백엔드 팀의 통합 부담을 낮춤

6.2 어려웠던 점

- ENUM 안 인라인 주석이 MySQL에서 파싱 오류를 일으키는 것을 실행 전 발견하지 못함 → SQL 리뷰 단계 필요
- 파일이 폴더 구조 없이 한 폴더에 섞이는 문제 → Docker 볼륨 마운트 시 폴더 구조 사전 확인 필요
- generate_seed.py의 SQL 청크 세미콜론 오류 → SQL 생성 스크립트는 실제 DB에서 검증 후 커밋 필요

6.3 남은 작업 (P2~P3)

- Redis 실제 연결 (현재 Mock 클라이언트 → 배포 시 실제 Redis 전환)
- Gemini API 키 적용 후 실제 AI 응답 품질 검증 및 프롬프트 튜닝
- EXPLAIN 분석 후 필요한 추가 인덱스 적용 (index_analysis.sql 참고)
- solved.ac API 크롤링 법적 검토 완료 후 실제 배치 수집 가동
- 포트폴리오 PDF 생성 기능 (weasyprint 또는 puppeteer)

전체 완료 현황

현재 상태 요약

DB/AI 영역 전체 완성 — 21개 파일, 5,700줄+, 39/39 테스트 통과
로컬(Docker) 환경에서 Django 서버 실행 및 MySQL 데이터 정상 연결 확인
백엔드 팀이 API 개발을 바로 시작할 수 있는 상태
orion 서버 배포는 SETUP_GUIDE.md 12단계 절차에 따라 진행 가능